Gamification React Native Integration Guide

Gamification React Native Integration Guide

Document Version - 1.0.1

Overview

Gamification is now available as part of the RN HydraLib package and enables gamified experiences within your application.

If you are already using the RN HydraLib package, no additional package integration is required.

Required Changes

1. Update to the Latest RN HydraLib Version

Update your application to the latest HydraLib version that is now publicly available.

To install the public Package

npm:

```shell
npm install @capillarytech/react-native-hydra-lib@beta
```

Yarn:

```shell
yarn add @capillarytech/react-native-hydra-lib@beta
```

Minimum Required Version

`1.7.0-beta.7 (current Beta version)`

---

2. Add Gamification Configuration

Add the following keys to Firebase Remote Config:

```
None
hydra_gamification_region

hydra_gamification_key

hydra_gamification_base_url

hydra_org_id
```

> ⚠️ IMPORTANT
> Key and url values are mandatory. HydraGamification will remain disabled if either value is missing.

---

3. Reporting Screens for Campaigns

To display Floating Button, Popup, and Bottom Sheet campaigns, report the current screen after sign-in:

```javascript
useFocusEffect(
  useCallback(() => {
    const timer = setTimeout(() => {
      Hydra.gamificationScreenAppeared('home');
    }, 500);

    return () => {
      clearTimeout(timer);
      Hydra.gamificationScreenDisappeared();
    };
  }, [])
);
```

> **Note:** The 500ms delay is required to allow the screen animation to complete before reporting.

4. Setting the Loyalty Token

To support the Loyalty View feature, set the Capillary B2B short-lived token:

```javascript
Hydra.setCapToken('token');
```

5. Banner View

Register a height listener and render the banner:

```javascript
const [bannerHeight, setBannerHeight] = useState(300);

useEffect(() => {
  const subscription =
    HydraNativeEventEmitter.getInstance()
      .addGamificationViewHeightChangedListener((height, entryPointId) => {
        if (height > 0 && entryPointId === BANNER_ID) {
          setBannerHeight(height);
        }
      });

  return () => {
    HydraNativeEventEmitter.getInstance().removeListener(subscription);
  };
}, []);
```

```none
<View style={styles.container}>
  <GamificationBannerView
    bannerId={BANNER_ID}
    style={[styles.banner, { height: bannerHeight }]}
  />
</View>
```

6. Embed View

```
None
<ScrollView style={styles.container}>
  <GamificationEmbedView
    embedId={EMBED_ID}
    style={[styles.embed, { height: embedHeight }]}
  />
</ScrollView>
```

7. Loyalty View

```
None
<ScrollView style={styles.container}>
  <GamificationLoyaltyView
    loyaltyId={LOYALTY_ID}
    style={[styles.loyalty, { height: loyaltyHeight }]}
  />
</ScrollView>
```

8. Callback for Analytic Event

For game events, we will fire this listener.

```
None
  useEffect(() => {
    const subscription =
      HydraNativeEventEmitter.getInstance().addGamificationAnalyticsEventListener(
        (name, payload) => {
          console.log('gamificationAnalyticsEvent: ', name, payload);
        }
      );
    return () => {
      HydraNativeEventEmitter.getInstance().removeListener(subscription);
    };
  }, []);
```

9. Callback for Deeplinks

When a click in the game needs to redirect to deeplink, this listener is fired.

```
None
useEffect(() => {
  const subscription =
    HydraNativeEventEmitter.getInstance().addGamificationDeeplinkListener(
      (url) => {
        console.log('gamificationDeeplink: ', url);
      }
    );
  return () => {
    HydraNativeEventEmitter.getInstance().removeListener(subscription);
  };
}, []);
```

10.Trigger on event

When you want to trigger a campaign on a custom event, use this:

```
None
Hydra.track('<YOUR_EVENT>');
```

Gamification WebSDK

Gamification WebSDK Integration Guide

Document Version - 1.0.0

Overview

Gamification is available as part of the cap-hydra-web-sdk package and enables gamified experiences within your web application.

If you are already using cap-hydra-web-sdk, no additional package installation is required.

Required Changes

1. Update to the Latest WebSDK Version

Update your application to the latest cap-hydra-web-sdk version that includes gamification support.

Minimum Required Version: 1.1.0-beta.20260619085241

```
npm install cap-hydra-web-sdk@beta
```

---

2. Add Gamification Configuration

Add the following keys to Firebase Remote Config (as fields inside your existing SDK config JSON):

Remote Config field	Description
gamificationWriteKey	Gamification write key — provided by the Capillary team

gamificationRegion	Gamification region code (e.g. "us", "me", “in”). Derives both the API base URL and stream URL automatically.
gamificationBaseUrl (Optional)	Gamification base url, will be provided by Capillary team. Right now it is an optional param.

IMPORTANT Both Remote Config values are mandatory. HydraGamification will remain disabled if either value is missing.

Example Remote Config JSON (add to your existing SDK config):

```
{
  "accountId": "YOUR_ACCOUNT_ID",
  "baseURL": "https://api.example.capillarytech.com",
  "orgId": "YOUR_ORG_ID",
  "gamificationWriteKey": "wk_live_xxxxxxxxxx",
  "gamificationRegion": "us",
  "gamificationBaseUrl": "https://api.gamification.capillarytech.com/"
}
```

Do not pass gamificationWriteKey **or** gamificationRegion **in** HydraSDK.initialize()**.** These values are fetched automatically from Remote Config at startup.

---

3. Campaign Display Rules — URL Path Targeting

The WebSDK determines campaign visibility using the **current page URL**. Path-based include/exclude rules are configured per entry point in the Gamification dashboard.

SPA navigation is handled automatically. The SDK observes URL changes via MutationObserver (with a 16 ms debounce) and re-evaluates campaign display rules on every route change — no code changes are required in your application.

IMPORTANT Campaign visibility and eligibility depend on the path rules configured in the Gamification dashboard. Ensure the correct URL patterns are set for each entry point by the Capillary team.

---

4. Set Capillary Token

A public API is available to support Loyalty View functionality.

Set the Capillary B2B short-lived token using:

```
HydraSDK.setCapToken("your-short-lived-token");
```

Call this method after the user logs in and whenever the token is refreshed.

IMPORTANT The Capillary token must be set before rendering the Loyalty View. Failure to provide a valid token may result in loyalty content not being displayed.

The token is:

- Persisted in IndexedDB (survives page reload — no need to set it on every load)
- Appended automatically as ?capToken=<value> to every loyalty iframe URL
- Cleared automatically on HydraSDK.reset() / logout

Automatic Experiences

Once configured, the following campaign types are automatically handled by the WebSDK:

Campaign type	**Behavior**
Floating Button	Fixed button (bottom-left) that slides up; opens a chat panel
Popup	Centered full-page modal with backdrop
Bottom Sheet	Sheet that slides up from the bottom

No additional implementation is required. The SDK automatically evaluates campaign eligibility and displays content when applicable.

Optional: Gamification UI Components

If you would like to display gamification content directly within your application screens, the following components are available.

For each component, place a placeholder with the matching ID anywhere in your page layout. The SDK detects when these elements appear in the DOM and renders content into them automatically — they do not need to exist at page load (SPA-safe).

Banner View

Place the placeholder wherever the banner should appear:

```
<div id="YOUR_BANNER_HOOK_ID"></div>
```

The hookId value is configured in the Gamification dashboard under the entry point settings.

Embedded View

Place the placeholder wherever the embedded widget should appear:

```
<div id="YOUR_EMBED_HOOK_ID"></div>
```

The hookId value is configured in the Gamification dashboard under the entry point settings.

Loyalty View

Place the placeholder wherever the loyalty view should appear:

```
<div id="YOUR_LOYALTY_EMBED_ID"></div>
```

The embedId value is provided by the Capillary team. This view requires a valid Capillary token — call HydraSDK.setCapToken(token) after the user logs in before the view renders.

Customer Integration Checklist

Required

- [ ] Install or update to cap-hydra-web-sdk version 1.1.0-beta.20260619085241 or later
- [ ] Add gamificationWriteKey to Firebase Remote Config JSON
- [ ] Add gamificationRegion to Firebase Remote Config JSON
- [ ] Publish the updated Remote Config
- [ ] Call HydraSDK.setCapToken(token) after login *(required for Loyalty View)*

Optional

- [ ] Add Banner View placeholder:
- [ ] Add Embedded View placeholder:
- [ ] Add Loyalty View placeholder:

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

For inline components (Banner, Embedded, Loyalty), place the corresponding placeholders in your HTML. The SDK renders content into them automatically after HydraSDK.identify() is called.

Hydra WEB SDK Integration Runbook

Capillary Hydra WebSDK — Integration Prompt

How to use this document: Copy everything inside the fenced block below (from `--- START ---` to `--- END ---`) and paste it directly into your AI chat. Replace the placeholder values (marked with `MY_*`) with your actual credentials from your Firebase project, then press Enter. Cursor will scan your project and integrate the SDK automatically.

--- START --- (copy from here)

Integrate Capillary Hydra WebSDK into my application

I want to integrate the **Capillary Hydra WebSDK** (`cap-hydra-web-sdk`) into my web application for event tracking, user identification, and analytics. Please follow every step below exactly.

Firebase Project Setup (First-time Setup)

If you don't have a Firebase project set up yet, follow these steps:

1. Create Firebase Project

1. Go to the Firebase Console
2. Click **"Add project"** or **"Create a project"**
3. Enter your project name (e.g., "my-app-analytics")
4. Choose whether to enable Google Analytics (recommended)
5. Select or create a Google Analytics account if enabled
6. Click **"Create project"**

2. Add Web App to Firebase Project

1. In your Firebase project dashboard, click the **Web icon** (`</>`) to add a web app
2. Enter your app nickname (e.g., "My Web App")
3. Check **"Also set up Firebase Hosting"** if you plan to use Firebase Hosting (optional)
4. Click **"Register app"**
5. **Copy the Firebase configuration object** - you'll need this for Step 2 below

The config object looks like this:

```javascript
const firebaseConfig = {
  apiKey: "AIzaSyC...",
  authDomain: "my-project.firebaseapp.com",
  databaseURL: "https://my-project-default-rtdb.firebaseio.com",
  projectId: "my-project",
  storageBucket: "my-project.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abcdef123456",
  measurementId: "G-XXXXXXXXXX", // optional, only if Analytics is enabled
};
```

3. Set up Remote Config

1. In your Firebase project, go to **"Remote Config"** in the left sidebar
2. Click **"Create configuration"**
3. Click **"Add parameter"**
4. Set **Parameter key** to your desired config key name (e.g., `hydra_sdk_config`)
5. Set **Default value** to a JSON object with your configuration values from the Capillary team (see **Configuration Reference** section below for the full JSON example and field details)
6. Click **"Save"** then **"Publish changes"**
7. **Remember your parameter key name** - you'll use this as `remoteConfigKey` in Step 2

4. Generate VAPID Key for Push Notifications

1. In your Firebase project, go to **"Project settings"** (gear icon in sidebar)
2. Click the **"Cloud Messaging"** tab
3. Scroll down to **"Web configuration"**
4. Under **"Web Push certificates"**, click **"Generate key pair"**
5. **Copy the generated VAPID key** - you'll need this for Step 2

5. Enable Required Firebase Services

Make sure these services are enabled in your Firebase project:

1. **Remote Config**: Go to Remote Config → should show your configuration
2. **Cloud Messaging**: Go to Cloud Messaging → should show dashboard
3. **Realtime Database** (if using `databaseURL`):
    - Go to Realtime Database → Click "Create Database"
    - Choose your region and security rules (start in test mode for development)

6. Get Firebase Messaging Service Worker

Contact the **Capillary team** to obtain the `firebase-messaging-sw.js` service worker file template. This file is required for push notifications to work properly.

Once you receive it:

1. Update the Firebase config inside the file with your credentials from step 2
2. Place it in your project's `public/` directory
3. Set `serviceWorkerPath` in your SDK config (Step 2) based on where your app is served:

Scenario	`serviceWorkerPath` value
App at root (`https://example.com/`)	`"/firebase-messaging-sw.js"`
App at subpath (`https://example.com/my-app/`)	`"/my-app/firebase-messaging-sw.js"`

Browser Support

Browser	Minimum Version	Notes
Chrome	49+	Full support
Firefox	52+ (ESR)	Full support
Safari	10+	Service Worker requires 11.1+
Edge	79+ (Chromium)	Full support
IE	11	Limited: no Service Worker, no Push API, no Web Locks

Step 1 — Install the SDK

I'll detect your package manager and install the SDK accordingly:

For npm projects (if `package-lock.json` exists):

```
Shell
npm install cap-hydra-web-sdk
```

For yarn projects (if `yarn.lock` exists):

```
Shell
yarn add cap-hydra-web-sdk
```

For pnpm projects (if `pnpm-lock.yaml` exists):

```
Shell
pnpm add cap-hydra-web-sdk
```

For bun projects (if `bun.lockb` exists):

```
Shell
bun add cap-hydra-web-sdk
```

If no lock file is detected, I'll use npm as the default package manager.

Step 2 — Create an SDK service file

Create a file at `src/services/sdkService.ts` (or `.js` if not using TypeScript). This file initializes the SDK once at **module load time** and exposes wrapper methods.

> **Important:** Follow my application's existing naming conventions and directory structure. If my codebase uses different patterns (e.g., `camelCase` vs `kebab-case` for filenames, different service directory names, different export patterns), adapt the file location and naming accordingly while keeping the same functionality.

Use this template and replace every `MY_*` placeholder with my actual credentials:

```
TypeScript
import { HydraSDK } from "cap-hydra-web-sdk";

class SDKService {
  initialize() {
    try {
```

```
      HydraSDK.initialize({
        firebaseConfig: {
          apiKey: "MY_FIREBASE_API_KEY",
          authDomain: "MY_PROJECT_ID.firebaseapp.com",
          databaseURL: "https://MY_PROJECT_ID-default-rtdb.firebaseio.com",
          projectId: "MY_PROJECT_ID",
          storageBucket: "MY_PROJECT_ID.appspot.com",
          messagingSenderId: "MY_SENDER_ID",
          appId: "MY_APP_ID",
          measurementId: "MY_MEASUREMENT_ID", // optional
        },
        remoteConfigKey: "MY_REMOTE_CONFIG_KEY",
        appVersion: "1.0.0", // my app's version string
        notifications: {
          vapidKey: "MY_VAPID_KEY",
          serviceWorkerPath: "/firebase-messaging-sw.js", // adjust if app is at a
subpath
        },
      });
    } catch (error) {
      console.error("Failed to initialize SDK:", error);
    }
  }

  // --- Page Views ---
  trackPageView(options?: {
    url?: string;
    title?: string;
    category?: string;
    properties?: Record<string, unknown>;
  }) {
    HydraSDK.pageView(options);
  }

  // --- Custom Events ---
  trackEvent(name: string, properties?: Record<string, unknown>) {
    HydraSDK.track(name, properties);
  }

  // --- User Identity ---
  identifyUser(
    identifierType: "email" | "mobile" | "externalId",
    cuid: string,
    actionType: "USER_REGISTER" | "USER_UPDATE",
    traits?: {
```

```
      firstName?: string;
      lastName?: string;
      gender?: string;
      birthday?: string;
      city?: string;
      country?: string;
      email?: string;
      mobile?: string;
      externalId?: string;
      custom?: Record<string, unknown>;
    },
  ) {
    HydraSDK.identify({
      identifier: { type: identifierType, cuid },
      actionType,
      traits,
    });
  }

  // --- Get User Info (anonymous users only) ---
  getUser() {
    return HydraSDK.getUser();
  }

  // --- Logout / Reset ---
  reset() {
    HydraSDK.reset();
  }

  // --- Cookie Consent ---
  enableTracking() {
    HydraSDK.enableSDK();
  }

  disableTracking() {
    HydraSDK.disableSDK();
  }
}

export const sdkService = new SDKService();

// IMPORTANT: Initialize at module load — NOT inside useEffect, ngOnInit, or
similar hooks
sdkService.initialize();
```

Key rules:

- `HydraSDK.initialize()` must be called at **module load time** (top level of the file), not inside React `useEffect`, Angular `ngOnInit`, or any async lifecycle hook.
- `initialize()` is safe to call multiple times — subsequent calls return the existing instance.
- All other SDK methods (`track`, `pageView`, `identify`, `reset`) internally wait for initialization to complete, so no `isInitialized()` guard is needed.
- Do NOT re-identify the user on page refresh. The SDK persists user context in IndexedDB automatically.
- All calls are fire-and-forget — they return immediately.
- Components should never import `WebSDK` directly — all calls go through this service.

Usage Notes

The SDK service file above exposes all available methods. Here are non-obvious details to keep in mind when integrating:

- **Page views:** `trackPageView()` with no arguments auto-captures the current URL and page title. Call it on route changes.
- **User identity:** Use `"USER_REGISTER"` as `actionType` for both login and signup. Use `"USER_UPDATE"` only for profile updates. Supported identifier types: `"email"`, `"mobile"`, `"externalId"`. Call `reset()` on logout.
- **Get user info:** `getUser()` returns a snapshot of the current visitor's identity, session, and engagement status (visitor ID, session ID, session number, first session flag, returning visitor flag). This method is only available for **anonymous users** — it throws an error if the user has been identified, to prevent exposure of identified-user data.
- **Cookie consent:** When tracking is disabled via `disableTracking()`, new events are dropped. Already-queued events are preserved in IndexedDB and sent when re-enabled. Persist the consent decision in your own storage and restore it on app load.

Configuration Reference

Required fields in `HydraSDK.initialize()`:

Field	Type	Description
`firebaseConfig`	`FirebaseConfig`	Firebase project configuration (from your Firebase console)

Field	Type	Description
`remoteConfigKey`	`string`	Remote Config key for SDK config (from your Firebase Remote Config)

Optional fields:

Field	Type	Default	Description
`appVersion`	`string`	—	Host app version for event metadata
`retryDelays`	`number[]`	`[60000, 120000, 240000, 480000, 900000]`	Custom retry delays in ms
`notifications`	`NotificationConfig`	—	Push notification config

Auto-populated from Remote Config (do NOT set manually):

Field	Required	Default	Description
`accountId`	Yes	—	Organization account identifier
`baseURL`	Yes	—	API base URL for all SDK requests
`orgId`	Yes	—	Organization ID for API authentication
`brandId`	No	—	Brand identifier for namespaced database names
`applicationId`	No	—	Application identifier for namespaced database names
`source`	No	`WEBPUSH`	Source identifier for visitor profiles

Field	Required	Default	Description
`channelType`	No	`web`	Communication channel type
`sessionTimeoutMinutes`	No	`30`	Session inactivity timeout in minutes
`batchSize`	No	`50`	Max events per batch request
`debugLevel`	No	`NONE`	Logging level (`NONE` / `ERROR` / `WARN` / `INFO` / `DEBUG`)

Example Remote Config JSON (stored as the value of your `remoteConfigKey` in Firebase Console):

```json
{
  "accountId": "100458",
  "baseURL": "https://api.example.capillarytech.com",
  "orgId": "100458",
  "brandId": "1",
  "applicationId": "2",
  "source": "WEBPUSH",
  "channelType": "web",
  "sessionTimeoutMinutes": 30,
  "batchSize": 50,
  "debugLevel": "NONE"
}
```

> **Note:** The required fields (`accountId`, `baseURL`, `orgId`) must exist in your Remote Config JSON. If missing, `HydraSDK.initialize()` will fail. Optional fields use their defaults if omitted.

Public API Quick Reference

Method	Description
`HydraSDK.initialize(config)`	Initialize the SDK singleton (call once)
`HydraSDK.track(eventName, properties?)`	Track a custom behavioral event
`HydraSDK.pageView(options?)`	Track a page view (auto-captures URL/title if no args)
`HydraSDK.identify(params)`	Identify a user (login / signup / update)
`HydraSDK.getUser()`	Get anonymous user's identity and session snapshot
`HydraSDK.reset()`	Sign out user, return to anonymous tracking
`HydraSDK.enableSDK()`	Re-enable tracking after disable
`HydraSDK.disableSDK()`	Disable tracking (new events are dropped)

Now please implement the following in my application:

Required Implementation:

- **Step 1**: Install the SDK package
- **Step 2**: Create the SDK service file following my existing codebase conventions (file naming, directory structure, import/export patterns)

Usage Notes (do NOT auto-implement):

- The "Usage Notes" section is reference only. I will integrate these methods myself where I see fit in my application flow.

After completing the implementation, include the following in your response:

1. A **Firebase Project Setup checklist** — list all the steps from the "Firebase Project Setup" section above so I know exactly what to set up if I haven't already (creating a Firebase project, adding a web app, setting up Remote Config, generating a VAPID key, enabling services, and obtaining the service worker file from the Capillary team). Do NOT create or auto-generate any of these — just provide the checklist as a guide.

2. A reminder to replace all `MY_*` placeholders in the service file with actual credentials.

--- END ---

Experience+ iOS

Gamification iOS Integration Guide

Overview

Gamification is now available as part of the **Hydra iOS SDK** and enables gamified experiences within your application.

If you are already using the Hydra iOS SDK, **no additional framework integration is required.**

HydraGamification is bundled within **HydraCore** and is enabled through configuration.

This guide covers integration for:

- Swift (UIKit)
- SwiftUI
- Objective-C

Required Changes

1. Update to the Latest Hydra SDK Version

Update your application to the latest Hydra iOS SDK version that includes HydraGamification support.

Repository URL:

```
None
https://github.com/Capillary/hydra-ios-sdk-spm.git
```

Minimum Required Version

1.7.5-preprod.10

---

2. Add Gamification Configuration

Add the following Firebase Remote Config keys:

```
None
hydra_gamification_region
hydra_gamification_key
hydra_gamification_base_url ()
```

Configure Hydra using these values during SDK initialization.

Swift

```
None
let configuration = try HydraConfiguration(
    ...
    gamificationWriteKey: writeKey,
    gamificationRegion: region,
    gamificationBaseURL: baseURL
)

let hydra = try Hydra.generate(with: configuration)
```

SwiftUI

Hydra configuration is identical to the Swift example.

Objective-C

Hydra initialization is performed through the existing Swift bootstrap used by your application.

URL Resolution

Hydra resolves the Gamification endpoint using the following priority:

1. If **hydra_gamification_base_url** is provided, it will be used as the Gamification base URL.
2. Otherwise, Hydra will resolve the base URL using **hydra_gamification_region**.

This allows existing integrations to continue using region-based configuration while supporting custom endpoints.

> ⚠️ **IMPORTANT**
>
> `hydra_gamification_key` is mandatory.
>
> Either `hydra_gamification_region` or `hydra_gamification_base_url` must be provided.
>
> HydraGamification will remain disabled if the required configuration is unavailable.

---

3. Report Current Screen

HydraGamification uses screen information to determine when and where campaigns should be displayed.

Swift (UIKit)

```
None
override func viewDidAppear(_ animated: Bool) {
    super.viewDidAppear(animated)

    hydra.gamificationScreenAppeared("SampleScreenName")
}

override func viewWillDisappear(_ animated: Bool) {
     super.viewWillDisappear(animated)
     hydra.gamificationScreenDisappeared("SampleScreenName")
}
```

SwiftUI

```
None
SampleView()
    .gamificationScreen("SampleScreenName")

Note:- The `gamificationScreen` modifier automatically reports
```

```
screen appearance and disappearance. No manual lifecycle callbacks
are required.
```

Objective-C

```
None
- (void)viewDidAppear:(BOOL)animated {
    [super viewDidAppear:animated];

    [HydraGamificationUI reportScreen:@"SampleScreenName"];
}
- (void)viewWillDisappear:(BOOL)animated {
     [super viewWillDisappear:animated];

    [HydraGamificationUI
reportScreenDisappeared:@"SampleScreenName"];

}
```

> ⚠️ **IMPORTANT**
>
> Screen reporting is mandatory for screen-targeted campaigns.
>
> Campaign visibility and eligibility depend on the currently reported screen.

---

4. Set Capillary Token

A new public API has been introduced to support Loyalty View functionality.

Set the Capillary B2B short-lived token before displaying a Loyalty View.

Swift

```
Swift
hydra.setCapToken(token)
```

SwiftUI

```Swift
hydra.setCapToken(token)
```

Objective-C

```None
[HydraGamificationUI setCapToken:token];
```

> ⚠️ **IMPORTANT**
>
> The Capillary token must be set before rendering the Loyalty View.
>
> Failure to provide a valid token may result in loyalty content not being displayed.

Automatic Experiences

Once configured, the following campaign types are automatically handled by Hydra.

- Floating Button Campaigns
- Popup Campaigns
- Bottom Sheet Campaigns

No additional implementation is required.

Hydra automatically evaluates campaign eligibility and displays content when applicable.

Optional: Gamification Delegate

HydraGamificationDelegate provides callbacks to the host application for gamification lifecycle events, campaign interactions, analytics events, deeplink requests, and event triggers.The host application can implement the delegate as follows:

Swift (UIKit)

Swift

```
// MARK: - Gamification Events

extension HydraBaseManager: HydraGamificationDelegate {

    func gamificationDidInitialize() {
        Logger.debug("[Gamification] initialized")
    }

    func gamificationDidFail(_ error: Error) {
        Logger.debug("[Gamification] failed: \(error)")
    }

    func gamificationDidLoadEntryPoints(
        _ entryPoints: [HydraGamificationEntryPoint]
    ) {
        Logger.debug(
            "[Gamification] entry points loaded: \(entryPoints.count)"
        )
    }

    func gamificationDidOpenCampaign(_ campaignId: String) {
        Logger.debug(
            "[Gamification] campaign opened: \(campaignId)"
        )
    }

    func gamificationDidCloseCampaign(_ campaignId: String) {
        Logger.debug(
            "[Gamification] campaign closed: \(campaignId)"
        )
    }

    func gamificationWidgetHeightChanged(
        entryPointId: String,
        height: CGFloat
    ) {
        Logger.debug(
```

```
            "[Gamification] widget height changed: \(height)"
        )
    }

    /// Web analytics events emitted from campaign content,
    /// such as BUTTON_CLICKED or PAGE_OPENED.
    func gamificationDidEmitAnalyticsEvent(
        _ name: String,
        payload: [String: Any]
    ) {
        Logger.debug(
            "[Gamification] analytics event from web: \(name),
payload=\(payload)"
        )
    }

    /// Called when a campaign requests the host application
    /// to open a deeplink.
    ///
    /// HTTP/HTTPS links are handled by the SDK. The host application
    /// can handle app-specific custom-scheme deeplinks here.
    func gamificationRequestedDeeplink(_ url: String) {
        Logger.debug(
            "[Gamification] deeplink requested: \(url)"
        )

        if let url = URL(string: url) {
            UIApplication.shared.open(
                url,
                options: [:],
                completionHandler: nil
            )
        }
    }

    /// Called when a gamification event is triggered.
    func gamificationEventTrigger(
        eventName: String,
        info: [String: Any]
    ) {
```

```
        Logger.debug(
            "[Gamification] event triggered: \(eventName), info: \(info)"
        )
    }
}
```

Gamification UI Suppression

The SDK also provides APIs to temporarily suppress automatically presented gamification UI, such as floating action buttons, campaign popups, and other auto-presented overlays.

Host Application API

Swift (UIKit)

```
None
/// Suppresses or resumes automatically presented gamification popups/overlays.

hydra.setGamificationSuppressed(true)
```

Optional: Gamification UI Components

If you would like to display gamification content directly within your application screens, the following UI components are available.

For each view:

- Pass a unique ID.
- Register a height callback.
- Update the view height dynamically.

Banner View

Swift (UIKit)

```Swift
let bannerView = hydra.gamificationBannerView(
    bannerId: BANNER_ID
) { height in
    self.bannerHeightConstraint.constant = height
}
```

SwiftUI

```None
GamificationBannerView(
    bannerId: BANNER_ID
)
```

Objective-C

```None
UIView *banner =
[HydraGamificationUI bannerViewWithBannerId:BANNER_ID
                                   onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Embedded View

Swift

```None
let embedView = hydra.gamificationEmbedView(
    embedId: EMBED_ID
) { height in
    self.heightConstraint.constant = height
}
```

SwiftUI

```
None
GamificationEmbedView(
    embedId: EMBED_ID
)
```

Objective-C

```
None
UIView *embed =
[HydraGamificationUI embedViewWithEmbedId:EMBED_ID
                                 onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Loyalty View

Swift

```
None
let loyaltyView = hydra.gamificationLoyaltyView(
    embedId: LOYALTY_ID
) { height in
    self.heightConstraint.constant = height
}
```

SwiftUI

```
None
GamificationLoyaltyView(
    embedId: LOYALTY_ID
)
```

Objective-C

```
None
UIView *loyalty =
[HydraGamificationUI loyaltyViewWithEmbedId:LOYALTY_ID
                                   onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Customer Integration Checklist

Required

- Update to the latest Hydra iOS SDK version.
- Add `hydra_gamification_region`.
- Add `hydra_gamification_key`.
- Add `hydra_gamification_base_url` (Optional).
- Report screen names.
- Set the Capillary token using `setCapToken(...)` (Required for Loyalty View).

Optional

- Add Banner View.
- Add Embedded View.
- Add Loyalty View.

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

If you choose to integrate the optional UI components, Banner, Embedded, and Loyalty experiences can also be displayed directly within your application screens.

Experience+ Android

Gamification Android Integration Guide

Overview

Gamification is now available as part of the Hydra Android SDK and enables gamified experiences within your application.

If you are already using the Hydra Android SDK, no additional framework integration is required. HydraGamification is bundled within HydraCore and is enabled through configuration.

This guide covers integration for:

- Kotlin
- Java

Required Changes

1. Update to the Latest Hydra SDK Version

Update your application to the latest Hydra Android SDK version that includes HydraGamification support.

Minimum Required Version

`1.9.22-beta01` (Beta)

---

2. Add Gamification Configuration

Add the following Firebase Remote Config keys:

- `hydra_gamification_key`
- `hydra_gamification_region`
- `hydra_gamification_base_url` ()

Configure Hydra using these values during SDK initialization.

Kotlin

Kotlin

```kotlin
val hydraConfig = HydraConfig.Builder(applicationContext, accountId)
    .baseURL(baseURL)
    .sslPublicKey(sslPublicKey)
    .apply {
        if (writeKey.isNotEmpty()) {
            gamificationWriteKey(writeKey)
            if (baseUrl.isNotEmpty()) {
                gamificationBaseUrl(baseUrl)
            }
            if (region.isNotEmpty()) {
                gamificationRegion(GamificationRegion.valueOf(region.uppercase()))
            }
        }
    }
    .build()

HydraAPI.initialize(applicationContext, hydraConfig)
```

Java

Java

```java
HydraConfig.Builder builder = new HydraConfig.Builder(getApplicationContext(),
accountId)
    .baseURL(baseURL)
    .sslPublicKey(sslPublicKey);

if (!writeKey.isEmpty()) {
    builder.gamificationWriteKey(writeKey);
    if (!baseUrl.isEmpty()) {
        builder.gamificationBaseUrl(baseUrl);
    }
    if (!region.isEmpty()) {

builder.gamificationRegion(GamificationRegion.valueOf(region.toUpperCase()));
    }
}

HydraConfig hydraConfig = builder.build();
HydraAPI.initialize(getApplicationContext(), hydraConfig);
```

---

3. Wire GamificationUIManager

After Hydra initialization, create and store the `GamificationUIManager` in your `Application` class. This wires up the callback and enables automatic campaign rendering.

Kotlin

Kotlin

```
class MyApplication : Application() {

    private var gamificationUIManager: GamificationUIManager? = null

    fun getGamificationUIManager(): GamificationUIManager? = gamificationUIManager

    private fun initializeHydraSDK() {
        // ... HydraConfig setup and HydraAPI.initialize() ...

        // Wire gamification UI manager if gamification is enabled
        HydraAPI.getDefaultInstance(applicationContext)?.getGamificationManager()
            ?.let { flowManager ->
                flowManager.callback = MyGamificationCallback()
                gamificationUIManager = GamificationUIManager(flowManager,
applicationContext)
            }
    }
}
```

Java

Java

```
public class MyApplication extends Application {

    private GamificationUIManager gamificationUIManager;

    public GamificationUIManager getGamificationUIManager() {
        return gamificationUIManager;
    }

    private void initializeHydraSDK() {
        // ... HydraConfig setup and HydraAPI.initialize() ...

        // Wire gamification UI manager if gamification is enabled
        HydraAPI hydra = HydraAPI.getDefaultInstance(getApplicationContext());
        if (hydra != null) {
```

```
            GamificationFlowManager flowManager = hydra.getGamificationManager();
            if (flowManager != null) {
                flowManager.setCallback(new MyGamificationCallback());
                gamificationUIManager = new GamificationUIManager(flowManager,
getApplicationContext());
            }
        }
    }
}
```

---

4. Report Current Screen

HydraGamification uses screen information to determine when and where campaigns should be displayed.

Register `ActivityLifecycleCallbacks` in your `Application.onCreate()` to automatically report screen changes for every Activity.

Kotlin

Kotlin

```
override fun onCreate() {
    super.onCreate()

    registerActivityLifecycleCallbacks(object : ActivityLifecycleCallbacks {
        override fun onActivityCreated(activity: Activity, savedInstanceState:
Bundle?) {}
        override fun onActivityStarted(activity: Activity) {}

        override fun onActivityResumed(activity: Activity) {
            gamificationUIManager?.onScreenChanged(activity,
activity.javaClass.simpleName)
        }

        override fun onActivityPaused(activity: Activity) {}
        override fun onActivityStopped(activity: Activity) {}
        override fun onActivitySaveInstanceState(activity: Activity, outState:
Bundle) {}

        override fun onActivityDestroyed(activity: Activity) {
```

```
            gamificationUIManager?.onActivityDestroyed(activity)
        }
    })

    // ... rest of initialization
}
```

Java

```
Java
@Override
public void onCreate() {
    super.onCreate();

    registerActivityLifecycleCallbacks(new ActivityLifecycleCallbacks() {
        @Override public void onActivityCreated(Activity activity, Bundle
savedInstanceState) {}
        @Override public void onActivityStarted(Activity activity) {}

        @Override
        public void onActivityResumed(Activity activity) {
            if (gamificationUIManager != null) {
                gamificationUIManager.onScreenChanged(activity,
activity.getClass().getSimpleName());
            }
        }

        @Override public void onActivityPaused(Activity activity) {}
        @Override public void onActivityStopped(Activity activity) {}
        @Override public void onActivitySaveInstanceState(Activity activity,
Bundle outState) {}

        @Override
        public void onActivityDestroyed(Activity activity) {
            if (gamificationUIManager != null) {
                gamificationUIManager.onActivityDestroyed(activity);
            }
        }
    });

    // ... rest of initialization
}
```

IMPORTANT

- Screen reporting is mandatory for screen-targeted campaigns.
- Campaign visibility and eligibility depend on the currently reported screen.
- `onActivityDestroyed` cleanup is required to prevent memory leaks from attached entry point views.

---

5. Handle App Foreground/Background Lifecycle

Register a `ProcessLifecycleOwner` observer to notify gamification when the app moves to foreground or background. This enables freshness checks.

Kotlin

```kotlin
ProcessLifecycleOwner.get().lifecycle.addObserver(object :
DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) {
        gamificationUIManager?.onAppForegrounded()
    }

    override fun onStop(owner: LifecycleOwner) {
        gamificationUIManager?.onAppBackgrounded()
    }
})
```

Java

```java
ProcessLifecycleOwner.get().getLifecycle().addObserver(new
DefaultLifecycleObserver() {
    @Override
    public void onStart(@NonNull LifecycleOwner owner) {
        if (gamificationUIManager != null) {
            gamificationUIManager.onAppForegrounded();
        }
    }

    @Override
    public void onStop(@NonNull LifecycleOwner owner) {
        if (gamificationUIManager != null) {
```

```
            gamificationUIManager.onAppBackgrounded();
        }
    }
});
```

---

6. Set Capillary Token

A public API is available to support Loyalty View functionality. Set the Capillary B2B short-lived token before displaying a Loyalty View.

Kotlin

```kotlin
val hydra = HydraAPI.getDefaultInstance(context)
hydra?.setCapToken(token)
```

Java

```java
HydraAPI hydra = HydraAPI.getDefaultInstance(context);
if (hydra != null) {
    hydra.setCapToken(token);
}
```

> **IMPORTANT**
>
> - The Capillary token must be set before rendering the Loyalty View.
> - Failure to provide a valid token may result in loyalty content not being displayed.

---

7. Handle Late Initialization via Remote Config

If gamification keys arrive via Firebase Remote Config after Hydra is already initialized, use `updateRemoteConfig` to pass the keys. The SDK will lazily initialize gamification internally.

After `updateRemoteConfig` succeeds, wire the `GamificationUIManager` if it hasn't been created yet.

Kotlin

```kotlin
val success = HydraAPI.updateRemoteConfig(
    orgId = newOrgId,
    accountId = newAccountId,
    baseURL = newBaseUrl,
    sslPublicKey = newSslKey,
    gamificationWriteKey = newCgWriteKey.takeIf { it.isNotEmpty() },
    gamificationRegion = cgRegion,
    gamificationBaseUrl = newCgBaseUrl
)

if (success && gamificationUIManager == null) {
    HydraAPI.getDefaultInstance(applicationContext)
        ?.getGamificationManager()
        ?.let { flowManager ->
            flowManager.callback = MyGamificationCallback()
            gamificationUIManager = GamificationUIManager(flowManager,
applicationContext)
        }
}
```

Java

```java
boolean success = HydraAPI.updateRemoteConfig(
    newOrgId,
    newAccountId,
    newBaseUrl,
    newSslKey,
    newCgWriteKey.isEmpty() ? null : newCgWriteKey,
    cgRegion,
    newCgBaseUrl
);

if (success && gamificationUIManager == null) {
    HydraAPI hydra = HydraAPI.getDefaultInstance(getApplicationContext());
    if (hydra != null) {
        GamificationFlowManager flowManager = hydra.getGamificationManager();
```

```
        if (flowManager != null) {
            flowManager.setCallback(new MyGamificationCallback());
            gamificationUIManager = new GamificationUIManager(flowManager,
getApplicationContext());
        }
    }
}
```

Automatic Experiences

Once configured, the following campaign types are automatically handled by Hydra:

- **Floating Button Campaigns**
- **Popup Campaigns**
- **Bottom Sheet Campaigns**

No additional implementation is required. Hydra automatically evaluates campaign eligibility and displays content when applicable.

Optional: Gamification Callback

Implement `GamificationCallback` to receive gamification lifecycle events.

Kotlin

```
Kotlin
class MyGamificationCallback : GamificationCallback {

    override fun onInitSuccess() {
        // Gamification initialized successfully
    }

    override fun onInitFailure(error: GamificationError) {
        // Handle initialization failure
    }

    override fun onEntryPointsLoaded(entryPoints: List<EntryPointData>) {
```

```
        // Entry points are ready
    }

    override fun onCampaignOpened(campaignId: String) {
        // A campaign was opened
    }

    override fun onCampaignClosed(campaignId: String) {
        // A campaign was closed
    }

    override fun onAnalyticsEvent(eventName: String, data: Map<String, Any?>) {
        // Analytics event received
    }
}
```

Java

Java

```
public class MyGamificationCallback implements GamificationCallback {

    @Override
    public void onInitSuccess() {
        // Gamification initialized successfully
    }

    @Override
    public void onInitFailure(@NonNull GamificationError error) {
        // Handle initialization failure
    }

    @Override
    public void onEntryPointsLoaded(@NonNull List<EntryPointData> entryPoints) {
        // Entry points are ready
    }

    @Override
    public void onCampaignOpened(@NonNull String campaignId) {
        // A campaign was opened
    }

    @Override
    public void onCampaignClosed(@NonNull String campaignId) {
```

```
        // A campaign was closed
    }

    @Override
    public void onAnalyticsEvent(@NonNull String eventName, @NonNull Map<String,
?> data) {
        // Analytics event received
    }
}
```

Optional: Gamification UI Components

If you would like to display gamification content directly within your application screens, the following UI components are available.

Banner View

Add to your XML layout. The `android:id` must correspond to the `bannerId` configured in the campaign dashboard.

```
XML
<com.capillary.hydra.gamification.ui.entrypoints.BannerView
    android:id="@+id/banner110"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Banner views are automatically bound and rendered by `GamificationUIManager` when `onScreenChanged` is called. No additional code is needed.

Embedded View

Add to your XML layout. The `android:id` must correspond to the `bannerId` configured in the campaign dashboard, same pattern as Banner View.

```
XML
<com.capillary.hydra.gamification.ui.entrypoints.EmbedWebView
    android:id="@+id/rn_embed1"
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content" />
```

Embedded views are automatically bound and rendered by `GamificationUIManager` when `onScreenChanged` is called, same as Banner views. No additional code is needed.

Loyalty View

Add to your XML layout. The `android:id` must match the `embedId` from the API response.

```xml
<com.capillary.hydra.gamification.ui.entrypoints.LoyaltyEmbedView
    android:id="@+id/embedLoyalty1"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Loyalty views are automatically rendered by `GamificationUIManager` when `onScreenChanged` is called, provided the Capillary token has been set via `setCapToken()`.

Integration Checklist

Required

- [ ] Update to the latest Hydra Android SDK version (`1.9.22-beta01` or later).
- [ ] Add `hydra_gamification_key` to Firebase Remote Config.
- [ ] Add `hydra_gamification_region` to Firebase Remote Config.
- [ ] Add `hydra_gamification_base_url` to Firebase Remote Config (Optional).
- [ ] Wire `GamificationUIManager` in your Application class after SDK initialization.
- [ ] Register `ActivityLifecycleCallbacks` to report screen names via `onScreenChanged`.
- [ ] Register `ProcessLifecycleOwner` observer for `onAppForegrounded` / `onAppBackgrounded`.
- [ ] Call `onActivityDestroyed` in lifecycle callbacks for cleanup.
- [ ] Set the Capillary token using `setCapToken(token)` (Required for Loyalty View).

Optional

- [ ] Implement `GamificationCallback` for lifecycle events.
- [ ] Add Banner View.
- [ ] Add Embedded View.
- [ ] Add Loyalty View.

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

If you choose to integrate the optional UI components, Banner, Embedded, and Loyalty experiences can also be displayed directly within your application screens.

Experience+ Flutter Integration Document

Experience+ Flutter

Gamification Flutter Integration Guide

Overview

Gamification is now available as part of the `capillary_hydra` Flutter package and enables gamified experiences within your application.

If you are already using the `capillary_hydra` package, **no additional package integration is required.**

Required Changes

1. Update to the Latest capillary_hydra Version

Update your application to the latest capillary_hydra version that includes HydraGamification support.

Minimum Required Version

1.1.0 (To be released)

---

2. Add Gamification Configuration

Add the following keys to Firebase Remote Config:

```
hydra_gamification_region
hydra_gamification_key
hydra_gamification_url ()
```

Once these keys are added to Remote Config, capillary_hydra will automatically pick them up — no additional Dart code is required.

> ⚠️ **IMPORTANT**
>
> `hydra_gamification_region` and `hydra_gamification_key` are mandatory.
>
> `hydra_gamification_url` is optional.
>
> HydraGamification will remain disabled if either mandatory value is missing.

---

3. Report Current Screen

HydraGamification uses screen information to determine when and where campaigns should be displayed.

Example

```
@override
void initState() {
    super.initState();
    Hydra.gamificationScreenAppeared(screenName: 'SampleScreenName');
}

@override
void dispose() {
    Hydra.gamificationScreenDisappeared();
    super.dispose();
}
```

For a screen that stays mounted underneath pushed routes (for example a persistent Home tab), `initState`/`dispose` will not fire when the screen regains focus. Use a `RouteObserver` with `RouteAware`, and also observe app lifecycle so the campaign re-evaluates when the app backgrounds/foregrounds while that screen is still current:

```
final RouteObserver<PageRoute<dynamic>> routeObserver =
        RouteObserver<PageRoute<dynamic>>();
```

```
// Register once on MaterialApp:
MaterialApp(navigatorObservers: [routeObserver], ...);

class _HomePageState extends State<HomePage>
        with RouteAware, WidgetsBindingObserver {
    bool _isCurrentRoute = true;

    @override
    void didChangeAppLifecycleState(AppLifecycleState state) {
        if (!_isCurrentRoute) return;
        if (state == AppLifecycleState.resumed) {
            Hydra.gamificationScreenAppeared(screenName:
'SampleScreenName');
        } else if (state == AppLifecycleState.paused) {
            Hydra.gamificationScreenDisappeared();
        }
    }

    @override
    void didPopNext() {
        _isCurrentRoute = true;
        Hydra.gamificationScreenAppeared(screenName:
'SampleScreenName');
    }

    @override
    void didPushNext() {
        _isCurrentRoute = false;
        Hydra.gamificationScreenDisappeared();
    }
}
```

> ⚠️ **IMPORTANT**
>
> Screen reporting is mandatory for screen-targeted campaigns.
>
> Campaign visibility and eligibility depend on the currently reported screen.

---

4. Set Capillary Token

A new public API has been introduced to support Loyalty View functionality.

Set the Capillary B2B short-lived token using the following method:

```
Hydra.setCapToken(token: 'token');
```

> ⚠️ **IMPORTANT**
>
> The Capillary token must be set before rendering the Loyalty View. Failure to provide a valid token may result in loyalty content not being displayed.

Automatic Experiences

Once configured, the following campaign types are automatically handled by capillary_hydra:

- Floating Button Campaigns
- Popup Campaigns
- Bottom Sheet Campaigns

No additional implementation is required.

capillary_hydra automatically evaluates campaign eligibility and displays content when applicable.

Optional: Gamification UI Components

If you would like to display gamification content directly within your application screens, the following widgets are available.

For Banner and Embedded views:

- Pass a unique ID.
- Register the `onHeightChanged` callback to update the height dynamically.

Banner View

```
double bannerHeight = 300;

SizedBox(
    height: bannerHeight,
    child: GamificationBannerView(
        bannerId: BANNER_ID,
        onHeightChanged: (height) {
            if (height > 0 && mounted) {
                setState(() => bannerHeight = height);
            }
        },
    ),
)
```

Embedded View

The same pattern can be followed for the Embedded view, using `GamificationEmbedView` and `embedId`.

```
SizedBox(
    height: embedHeight,
    child: GamificationEmbedView(
        embedId: EMBED_ID,
        onHeightChanged: (height) {
            if (height > 0 && mounted) {
                setState(() => embedHeight = height);
            }
        },
    ),
)
```

Loyalty View

Unlike Banner and Embedded views, `GamificationLoyaltyView` never reports a height change — size it explicitly with a fixed-height parent.

```
SizedBox(
```

```
    height: 611,
    child: GamificationLoyaltyView(loyaltyId: LOYALTY_ID),
)
```

> ⚠️ **IMPORTANT**
>
> Call `Hydra.setCapToken(...)` before mounting `GamificationLoyaltyView` — the native SDK cannot bind the Loyalty view without a cap token already set.

Customer Integration Checklist

Required

- Update to capillary_hydra version **1.1.0(next release)** or later.
- Add `hydra_gamification_region` to Firebase Remote Config.
- Add `hydra_gamification_key` to Firebase Remote Config.
- Report screen names.
- Set the Capillary token using `Hydra.setCapToken(...)` (Required for Loyalty View).

Optional

- Add Banner View.
- Add Embedded View.
- Add Loyalty View.

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

If you choose to integrate the optional UI components, Banner, Embedded, and Loyalty experiences can also be displayed directly within your application screens.

Web SDK Doc without firebase integration

WEB Hydra SDK - Integration Guide (Firebase excluded)

Table of Contents

1. Prerequisites
2. Installation
3. SDK Initialization
4. Identity Management
5. Event Tracking
6. Gamification
7. Optional Features
8. Configuration Reference

---

1. Prerequisites

Before you begin, register your app with the Capillary team. Once registered, the following configuration values will be provided:

Parameter	**Description**
Account ID	Your unique account identifier
Base URL	API base endpoint
Org ID	Your organization identifier
Gamification Base URL	Base URL for gamification services
Gamification Key	writeKey for gamification
Gamification Region	region value

---

2. Installation

Install the SDK package:

```shell
npm install cap-hydra-web-sdk
```

Note : For UAT purpose please use this package

```shell
npm install cap-hydra-web-sdk@beta
```

---

3. SDK Initialization

Call `HydraSDK.initialize()` **exactly once** at module load time. Do not call it inside lifecycle hooks such as `useEffect`, `ngOnInit`, or similar.

```javascript
import { HydraSDK } from 'cap-hydra-web-sdk';

HydraSDK.initialize({
  accountId: '<ACCOUNT_ID>',
  baseURL: '<BASE_URL>',
  orgId: '<ORG_ID>',
  gamificationBaseUrl : '<GAMIFICATION_BASE_URL>'
  gamificationWriteKey: '<GAMIFICATION_WRITE_KEY>',
  gamificationRegion: '<GAMIFICATION_REGION>',
});
```

> **Note** : Please don't pass anything to `firebaseConfig` or `remoteConfigKey` — omitting them tells the SDK to use the directly supplied configuration. If push notifications are needed, supply `firebaseConfig` inside the `notifications` block only (see Push Notifications).

---

4. User Sign-In (Identify)

Sign-In

Call `HydraSDK.identify()` when a user tried to sign in . Use `actionType: "USER_REGISTER"` for both cases:

```
JavaScript
HydraSDK.identify({
  identifier: { type: 'email' | 'mobile' | 'externalId', cuid: <corresponding value> },
  actionType: 'USER_REGISTER',
  traits: {
    firstName?: string;
    lastName?: string;
    gender?: string;
    birthday?: string;
    city?: string;
    country?: string;
    timezone?: string;
    email?: string;
    mobile?: string;
    externalId?: string;
    custom?: Record<string, unknown>;
  },
});
```

Note : traits are optional parameter.

Profile Update

To update user details after sign-in, use `actionType: "USER_UPDATE"`:

```
JavaScript
HydraSDK.identify({
  identifier: { type: 'email' | 'mobile' | 'externalId', cuid: <corresponding value> },
  actionType: 'USER_UPDATE',
  traits: {
    firstName?: string;
    lastName?: string;
    gender?: string;
    birthday?: string;
```

```
    city?: string;
    country?: string;
    timezone?: string;
    email?: string;
    mobile?: string;
    externalId?: string;
    custom?: Record<string, unknown>;
  },
});
```

Note : traits are optional parameter.

Sign-Out

Call `HydraSDK.reset()` when the user signs out. This clears identity, persisted user context, and any cap token, returning tracking to anonymous:

```
JavaScript
HydraSDK.reset();
```

---

5. Event Tracking

Custom Events

Track behavioral events using `WebSDK.track()`:

```
JavaScript
WebSDK.track(EVENT_NAME, {...ATTRIBUTES})
```

Example :

```
JavaScript
WebSDK.track('add_to_cart', {
  productId: 'SKU-123',
  quantity: 2,
});
```

Page Views (Optional)

Track page views using `WebSDK.pageView()`. If called without arguments, the SDK auto-captures the current URL and page title:

```
JavaScript
WebSDK.pageView({...OPTIONS});
```

Example :

```
JavaScript
WebSDK.pageView(options?: {
    url?: string;
    title?: string;
    category?: string;
    properties?: Record<string, unknown>;
  });
```

---

6. Gamification

Gamification is **disabled by default**. It activates only when both `gamificationWriteKey` and `gamificationRegion` are passed in the `initialize()` config. If either is missing, the module is silently skipped and the rest of the SDK is unaffected.

How It Works

Once enabled, gamification content renders **automatically after `HydraSDK.identify()` is called** — no separate "start gamification" call is needed.

- **Campaign eligibility** is evaluated against the current page URL, using include/exclude path rules configured per entry point in the gamification dashboard (exact URL, wildcard, or regex-literal formats).
- **SPA navigation** is handled automatically — the SDK watches for URL changes via `MutationObserver` and re-evaluates display rules on every route change. No route-change code is required in the host app.
- **Path rules** are configured by the Capillary team per entry point in the dashboard, not by the host app.

Automatic Campaigns (No Code Required)

These campaign types are appended to `document.body` automatically. No placeholder elements or additional code is needed.

Campaign Type	Behavior
Floating Button	Fixed button (bottom-left) that slides up; tapping it opens a chat panel.
Popup	Centered full-page modal with a semi-transparent backdrop.
Bottom Sheet	Sheet that slides up from the bottom of the screen, with a backdrop.

Inline Components (Require a Placeholder `<div>`)

For inline gamification content, add a placeholder `<div>` with the matching `id` in your HTML. The SDK watches the DOM and renders into it as soon as it appears — the element does **not** need to exist at page load, making this safe for SPA routes that mount later.

Banner View:

```
HTML

<div id="{hookId}"></div>
```

Embedded View:

```
HTML

<div id="{hookId}"></div>
```

> The `hookId` for both Banner and Embedded views is configured per entry point in the gamification dashboard.

Loyalty View:

```HTML
<div id="{embedId}"></div>
```

> The `embedId` is provided by the Capillary team. Loyalty View requires a valid Capillary token — call `HydraSDK.setCapToken(token)` after login, before the view is expected to render (see Loyalty Embeds).

---

7. Optional Push Feature

Push Notifications

If push notifications are used, supply the Messaging instance inside the `notifications` block during initialization:

```JavaScript
import { initializeApp } from "firebase/app";
import { getMessaging } from "firebase/messaging";

const firebaseApp = initializeApp
      (
            { apiKey: "MY_FIREBASE_API_KEY", /* ...rest of Firebase config */ },
            "hydra-fcm",
      );

HydraSDK.initialize({
  accountId: '<ACCOUNT_ID>',
  baseURL: '<BASE_URL>',
  orgId: '<ORG_ID>',
  notifications: {
    vapidKey: '<VAPID_KEY>',
    serviceWorkerPath: '/firebase-messaging-sw.js',
    messaging: getMessaging(firebaseApp),
  },
});
```

Requirements:

- `firebase-messaging-sw.js` must be deployed at the site root.
- VAPID key must be generated in the Firebase Console.

Loyalty Embeds

To support Loyalty View, set the Capillary B2B short-lived token after login:

```javascript
HydraSDK.setCapToken('<CAP_TOKEN>');
```

To retrieve the current token:

```javascript
const token = HydraSDK.getCapToken();
```

The token is appended automatically as `?capToken=<value>` to every loyalty iframe URL.

Cookie Consent / GDPR

Wire these methods into your app's consent flow to enable or disable the SDK based on user consent:

```javascript
// When user grants consent
HydraSDK.enableSDK();

// When user revokes consent
HydraSDK.disableSDK();
```

Anonymous Visitor Lookups

To retrieve the anonymous visitor identity before a user has been identified:

```javascript
// Returns anonymous ID (safe to call at any time)
const anonymousId = HydraSDK.getAnonymousId();

// Returns full anonymous user object (throws if user is already identified)
const user = HydraSDK.getUser();
```

---

8. Configuration Reference

All `initialize()` Fields

Field	Required?	Default	Description
`accountId`	**Yes**	—	Your unique account identifier
`baseURL`	**Yes**	—	API base endpoint
`orgId`	**Yes**	—	Your organization identifier
`gamificationWriteKey`	**Yes**	—	Gamification write key
`gamificationRegion`	**Yes**	—	Region value
`gamificationBaseUrl`	**Yes**	—	Override for gamification API base URL
`brandId`	No	—	Brand identifier
`applicationId`	No	—	Application identifier
`source`	No	`WEBPUSH`	Source channel
`channelType`	No	`web`	Channel type
`sessionTimeoutMinutes`	No	`30`	Session timeout in minutes
`batchSize`	No	`50`	Event batch size
`debugLevel`	No	`NONE`	Debug log level

SDK Methods Summary

Method	When to Call
`HydraSDK.initialize(config)`	Once at module load time

Method	When to Call
`HydraSDK.identify(params)`	On sign-in, sign-up, or profile update
`HydraSDK.reset()`	On sign-out
`HydraSDK.track(event, props?)`	To track custom behavioral events
`HydraSDK.pageView(options?)`	To track page views
`HydraSDK.setCapToken(token)`	After login, if using Loyalty View
`HydraSDK.getCapToken()`	To retrieve the current cap token
`HydraSDK.enableSDK()`	When user grants consent (GDPR)
`HydraSDK.disableSDK()`	When user revokes consent (GDPR)
`HydraSDK.getAnonymousId()`	To get anonymous visitor ID
`HydraSDK.getUser()`	To get anonymous user object (pre-identify only)

Content Security Policy (CSP) Configuration

If your application uses a Content Security Policy (CSP), allowlist the following domains so the SDK works correctly. Add these to your existing directives rather than replacing them.

connect-src
https://*.capillarytech.com
https://*.googleapis.com (only if using Remote Config or push notifications)
https://*.firebaseio.com (only if using Remote Config or push notifications)
https://*.firebaseapp.com (only if using Remote Config or push notifications)
https://*.posthog.com

script-src
'self'
https://*.gstatic.com (push notifications only)
https://*.posthog.com

frame-src
https://*.capillarytech.com

img-src
https://*.capillarytech.com

worker-src
'self'
blob:
data:

Example CSP Configuration

connect-src 'self' https://*.capillarytech.com https://*.googleapis.com https://*.firebaseio.com https://*.firebaseapp.com https://*.posthog.com;
script-src 'self' https://*.gstatic.com https://*.posthog.com;
frame-src 'self' https://*.capillarytech.com;
img-src 'self' https://*.capillarytech.com;
worker-src 'self' blob: data:;

If your application already has CSP directives configured, add these domains to the existing values rather than replacing them. If you're not using Remote Config or push notifications, you only need the capillarytech.com and posthog.com entries.

For any questions or support, please reach out to the Capillary team.

React native SDK doc without firebase integration

React Native Hydra SDK - Integration Guide (Firebase excluded)

Table of Contents

1. Prerequisites
2. Setup for React Native
3. Setup for iOS
4. Setup for Android
5. React Native Implementation
6. Log Configuration
7. API Reference

Prerequisites

Before you begin, register your app with the Capillary team. You will need to provide the following credentials:

Credential	**Platform**
Bundle ID	iOS
Application ID	Android
App Signature (SHA256)	Android (from your app signing report)

Once registered, the Capillary team will provide the following configuration values:

Parameter	**Description**
Account ID	Your unique account identifier
Base URL	API base endpoint
SSL Public Key	Public key for secure communication
Org ID	Your organization identifier

Parameter	Description
Gamification Base URL	Base URL for gamification services
Gamification Key	writeKey for gamification
Gamification Region	region value

Setup for React Native

Step 1: Install the Package

npm:

```
Shell
npm install @capillarytech/react-native-hydra-lib@beta
```

Yarn:

```
Shell
yarn add @capillarytech/react-native-hydra-lib@beta
```

Step 2: Add Hydra Configuration

Create `<PROJECT_ROOT>/capillary/hydra-config.json` with the following content:

hydra-config.json sample

```
JSON
{
  "console_log_level": 4,
  "enable_in_app_messaging": false,
  "enable_inbox_api": false,
  "enable_inbox_ui": false,
  "use_default_inbox_ui_config": false,
  "max_notification_count": 0,
  "enable_push_notification": false,
  "enable_hydra_sdk_firebase_registration": false,
  "hydra_notification_color": "#FF03DAC5",
```

```
  "enable_in_app_messaging_in_react_native": false
}
```

Setup for iOS

Step 1: Update the Podfile

In `ios/Podfile`, make the following two changes:

a) Add these lines just below the existing `require` block for React Native:

```
None
require Pod::Executable.execute_command('node', ['-p',
  'require.resolve(
    "@capillarytech/react-native-hydra-lib/scripts/HydraPods.rb",
    {paths: [process.argv[1]]},
  )', __dir__]).strip
```

b) In your main target, just below `config = use_native_modules!`, add:

```
None
hydra_pods
```

Step 2: Install Pods

Navigate to the `ios` folder and run:

```
Shell
pod install
```

Step 3: Initialize the Hydra SDK

In your `AppDelegate`, add the import statement at the top:

```swift
import react_native_hydra_lib
```

Then, in the `application:didFinishLaunchingWithOptions:` method, replace the return statement with the following:

Swift:

```swift
func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey:
Any]? = nil
) -> Bool {

  let result = super.application(application, didFinishLaunchingWithOptions:
launchOptions)

  let config = HydraNativeRemoteConfig(
    accountId: "<ACCOUNT_ID>",
    baseUrl: "<BASE_URL>",
    sslPublicKey: "<SSL_PUBLIC_KEY>",
    remoteLogLevel: 0,
    orgId: "<ORG_ID>",
    gamificationKey: "<GAMIFICATION_KEY>",
    gamificationRegion: "<GAMIFICATION_REGION>",
    gamificationBaseUrl: "<GAMIFICATION_BASE_URL>"
  )

  Hydra
    .sharedInstance()
    .initialize(
      application,
      didFinishLaunchingWithOptions: launchOptions ?? [:],
      with: nil,
      withHydraRemoteConfig: config
    )

  return result
}
```

Objective-C:

Use the equivalent Objective-C method signature:

```
None
- (BOOL)application:(UIApplication *)application
    didFinishLaunchingWithOptions:(NSDictionary *)launchOptions
```

> **Note:** Replace all placeholder values (e.g., `<ACCOUNT_ID>`, `<BASE_URL>`) with the credentials provided during registration. See Log Configuration for details on log levels.

Setup for Android

Step 1: Initialize the Hydra SDK

In your `MainApplication` file, add the following imports:

```
Kotlin
import com.hydralib.nativeapis.Hydra
import com.hydralib.nativeapis.HydraRemoteConfig
```

Add the following code to the `onCreate` function, after the default React Native generated code:

```
Kotlin
val config = HydraRemoteConfig(
  accountId = "<ACCOUNT_ID>",
  baseURL = "<BASE_URL>",
  sslPublicKey = "<SSL_PUBLIC_KEY>",
  remoteDebugLevel = 0,
  orgId = "<ORG_ID>",
  gamificationKey = "<GAMIFICATION_KEY>",
  gamificationRegion = "<GAMIFICATION_REGION>",
  gamificationBaseUrl = "<GAMIFICATION_BASE_URL>"
)

Hydra.initialize(this, remoteConfig = config)
```

> **Note:** Replace all placeholder values with the credentials provided during registration.

React Native Implementation

Checking SDK Initialization

Before using any APIs from the package, verify that the Hydra SDK has been initialized successfully:

```javascript
HydraCore.isSdkInitialized()
  .then((isInitialized) => {
    console.log('Hydra Instance Status: ', isInitialized);

    if (isInitialized) {
      // SDK is ready - proceed with your logic
    } else {
      // Register for the initialization callback
      const subscription =
        HydraNativeEventEmitter.getInstance()
          .addHydraSdkInitializationResultListener((result, message) => {
            // Handle initialization result
          });
    }
  })
  .catch((error) => {
    console.error('Error checking Hydra initialization status:', error);
  });
```

User Sign-In (Identify)

To use the Hydra SDK, first sign in the user using the `identify` API. You must pass `customerId`; other user details are optional. See the HydraUserDetails reference for more details.

```javascript
const userDetails = new HydraUserDetails(customerId);
await Hydra.identify(userDetails);
```

User Sign-Out (Reset)

```JavaScript
await Hydra.reset(customerId);
```

Reporting Screens for Campaigns

To display Floating Button, Popup, and Bottom Sheet campaigns, report the current screen after sign-in:

```JavaScript
useFocusEffect(
  useCallback(() => {
    const timer = setTimeout(() => {
      Hydra.gamificationScreenAppeared('home');
    }, 500);

    return () => {
      clearTimeout(timer);
      Hydra.gamificationScreenDisappeared();
    };
  }, [])
);
```

> **Note:** The 500ms delay is required to allow the screen animation to complete before reporting.

Setting the Loyalty Token

To support the Loyalty View feature, set the Capillary B2B short-lived token:

```JavaScript
Hydra.setCapToken('token');
```

Banner View

Register a height listener and render the banner:

JavaScript

```
const [bannerHeight, setBannerHeight] = useState(300);

useEffect(() => {
  const subscription =
    HydraNativeEventEmitter.getInstance()
      .addGamificationViewHeightChangedListener((height, entryPointId) => {
        if (height > 0 && entryPointId === BANNER_ID) {
          setBannerHeight(height);
        }
      });

  return () => {
    HydraNativeEventEmitter.getInstance().removeListener(subscription);
  };
}, []);
```

None

```
<View style={styles.container}>
  <GamificationBannerView
    bannerId={BANNER_ID}
    style={[styles.banner, { height: bannerHeight }]}
  />
</View>
```

Embed View

None

```
<ScrollView style={styles.container}>
  <GamificationEmbedView
    embedId={EMBED_ID}
    style={[styles.embed, { height: embedHeight }]}
  />
</ScrollView>
```

Loyalty View

None

```
<ScrollView style={styles.container}>
  <GamificationLoyaltyView
    loyaltyId={LOYALTY_ID}
```

```
    style={[styles.loyalty, { height: loyaltyHeight }]}
  />
</ScrollView>
```

Callback for Analytic Event

For game events, we will fire this listener.

```
None
  useEffect(() => {
    const subscription =
      HydraNativeEventEmitter.getInstance().addGamificationAnalyticsEventListener(
        (name, payload) => {
          console.log('gamificationAnalyticsEvent: ', name, payload);
        }
      );
    return () => {
      HydraNativeEventEmitter.getInstance().removeListener(subscription);
    };
  }, []);
```

Callback for Deeplinks

When a click in the game needs to redirect to deeplink, this listener is fired.

```
None
useEffect(() => {
  const subscription =
    HydraNativeEventEmitter.getInstance().addGamificationDeeplinkListener(
      (url) => {
        console.log('gamificationDeeplink: ', url);
      }
    );
  return () => {
    HydraNativeEventEmitter.getInstance().removeListener(subscription);
  };
}, []);
```

Trigger on event

When you want to trigger a campaign on a custom event, use this:

```
None
Hydra.track('<YOUR_EVENT>');
```

Set theme mode

When you want to set the theme, check available modes in API Reference below for ThemeMode enum:

```
TypeScript
Hydra.setThemeMode(ThemeMode.dark)
     .then(() => {
       console.log('Theme mode set to dark');
     })
     .catch((error) => {
       console.error('Error setting theme mode:', error);
     });
```

Get theme mode

When you want to set the theme, check available modes in API Reference below for ThemeMode enum:

```
TypeScript
Hydra.getThemeMode()
     .then((mode) => {
       console.log('Current theme mode is', mode);
     })
     .catch((error) => {
       console.error('Error getting theme mode:', error);
     });
```

Enable/Disable Hydra Gamification

You can disable(suppress) hydra gamification by passing true below, pass false to enable it.

TypeScript

```
Hydra.setGamificationSuppressed(true)
     .then(() => {
       console.log('Gamification suppressed set to true');
     })
     .catch((error) => {
       console.error('Error setting gamification suppressed:', error);
     });
```

Get Hydra Gamification status

You can check if hydra gamification is disabled(suppressed).

TypeScript

```
Hydra.isGamificationSuppressed()
     .then((suppressed) => {
       console.log('Gamification suppressed is', suppressed);
     })
     .catch((error) => {
       console.error('Error getting gamification suppressed:', error);
     });
```

Log Configuration

You can configure log levels in `hydra-config.json` (Android) and `HydraConfig.plist` (iOS).

There are two types of log levels:

Log Type	**Description**
Console Log Level	Logs output to Xcode console (iOS) and Logcat (Android)
Remote Log Level	Logs saved to the Capillary backend

Both values must be an integer between **0** and **5**:

Level	**Name**	**Description**
0	Off	Logging disabled
1	Error	Error-level messages only
2	Warn	Warnings and above
3	Info	Informational messages and above
4	Debug	Debug messages and above
5	Verbose	All messages

Configuration keys:

- **iOS:** Use the `LOG_LEVELS` key
- **Android:** Use the `log_levels` key

API Reference

HydraNativeEventEmitter

Get Singleton Instance

```
JavaScript
static getInstance(): HydraNativeEventEmitter;
```

Returns the singleton instance of `HydraNativeEventEmitter` used to invoke all other methods.

```
JavaScript
HydraNativeEventEmitter.getInstance();
```

Remove a Listener

```
JavaScript
HydraNativeEventEmitter.removeListener(
  emitterSubscription: EmitterSubscription
): void;
```

Parameter	Type	Required	Description
`emitterSubscription`	`EmitterSubscription`	Yes	The subscription object returned when the listener was added

Example:

```
JavaScript
HydraNativeEventEmitter.getInstance().removeListener(inboxUpdatedSubscription);
```

HydraUserDetails

Model class used for signing up, signing in, and updating user details.

Constructor:

```
JavaScript
new HydraUserDetails(
  customerId: string,
  firstName?: string | null,
  lastName?: string | null,
  email?: string | null,
  phone?: string | null
): HydraUserDetails
```

Parameter	Type	Required	Description
`customerId`	`string`	Yes	Can be mobile, email, InTouchUserId, externalId, or FCMToken. Determined by the **Account Linking Identifier** configured in the Capillary CRM dashboard under Organisation Settings > Omni Channel Settings > Channel Configurations > MAPP_SDK.
`firstName`	`string`	No	First name of the user
`lastName`	`string`	No	Last name of the user
`email`	`string`	No	Email address of the user
`phone`	`string`	No	Phone number of the user

Example:

```
JavaScript
const userDetails = new HydraUserDetails(
  '8888888888',
  'FirstName',
  'LastName',
  'hydra.test@capillarytech.com',
  '918888888888'
);
```

HydraLogLevel

Enumeration defining constants for console and remote log levels. Each higher level includes all levels below it.

TypeScript

```
enum HydraLogLevel {
  off     = 0,
  error   = 1,
  warn    = 2,
  info    = 3,
  debug   = 4,
  verbose = 5,
}
```

ThemeMode

Enumeration defining constants for ThemeMode.

TypeScript

```
export enum ThemeMode {
 none = 'NONE',
 auto = 'AUTO',
 light = 'LIGHT',
 dark = 'DARK',
}
```

For any questions or support, please reach out to the Capillary team.

iOS SDK Integration Doc (without firebase)

Hydra iOS SDK Integration Doc (without firebase)

Prerequisites

Before you get started, you need to register your app with the Capillary team. For registration, You need to provide bundle id for your iOS App:

Our app team will register the app with our system and will provide you below details:

1. Account Id
2. Base url
3. SSL public key
4. Org Id
5. Gamification Key
6. Gamification Region
7. Gamification URL (Optional)

You will also need to get the access for below github repositories from our access team:

1. https://github.com/Capillary/hydra-sdk-ios-packages

Installation

Access to Hydra SDK Library

The Hydra SDK is hosted in a private GitHub repository. You must have either:

- Valid GitHub credentials with access to the hydra-sdk-ios-packages repository.
- A GitHub username and Personal Access Token (PAT) with access to the repository.

Using GitHub Credentials

1. Open Xcode.
2. Navigate to Xcode → Settings (Preferences) → Accounts.
3. Add your GitHub account.
4. Ensure the account has access to the Capillary/hydra-sdk-ios-packages repository.

Using GitHub Username and Personal Access Token

Repository URL:

https://<GITHUB_USERNAME>:<PERSONAL_ACCESS_TOKEN>@github.com/Capillary/hydra-sdk-ios-packages.git

Installation Methods

You can install using any of the following approach

- Swift Package Manager (Package.swift)
- Xcode Package Manager
- CocoaPods

Install Using Swift Package Manager
Add the following dependency to your Package.swift:

```
None
import PackageDescription

 let package = Package(
     name: "<ProjectName>",
     dependencies: [
         .package(
             url:
"https://github.com/Capillary/hydra-sdk-ios-packages",
             .upToNextMajor(from: "1.1.0")
         )
     ]
 )
```

Install Using Xcode

1. Open your project in Xcode.
2. Select File → Add Packages…
3. Enter the repository URL (or the PAT URL if applicable).
4. Choose the desired version or branch.
5. Click Add Package.
6. Select the required Hydra frameworks.
7. Click Add Package again.
8. Import the required framework (for example, import HydraCore).

Install Using CocoaPods

```
None
platform :ios, '13.0'

 use_frameworks!

 target 'YourAppTargetName' do
   private_repo =
'https://<GITHUB_USERNAME>:<PERSONAL_ACCESS_TOKEN>@github.com/Capil
lary/hydra-sdk-ios-packages.git'

   pod 'HydraCore', :git => private_repo
   pod 'HydraPushNotification', :git => private_repo
   pod 'HydraPushNotificationServiceExtension', :git =>
private_repo
   pod 'HydraInAppMessage', :git => private_repo
   pod 'HydraInboxCore', :git => private_repo
   pod 'HydraInboxUI', :git => private_repo
 end

 target 'YourNotificationServiceExtensionTargetName' do
   pod 'HydraPushNotificationServiceExtension', :git =>
private_repo
 end
```

Run 'pod install' and open the generated .xcworkspace.

1. Hydra SDK Version

Add the Hydra iOS SDK version in your application.

Required Version

1.7.5-preprod.12 (Beta for UAT/POC)

---

2. SDK Initialization

Swift

```
None
 do {
 let hydraConfig = try HydraConfiguration(
      accountID: accountID,
      server: .genericServer(apiBaseURL, sslKey, [0, 5, 10, 30,
120]),
      orgID: orgId,
      gamificationWriteKey: gamificationWriteKey,
      gamificationRegion: gamificationRegion,
      gamificationBaseURL: gamificationBaseURL)

hydra = try Hydra.generate(with: hydraConfig, remoteLogLevel:
.debug)

} catch let error {
      Logger.error(error.localizedDescription)
}
```

3. User Sign-in

Swift (UIKit)

```
None
let userDetails = UserDetails(customerId: userName, firstName:
firstName, lastName: lastName, email: email, phone: phone)

hydra.reportUserSignIn(data: userDetails, customData: customData)
```

4. User Sign-out

Swift (UIKit)

```
None
let customAttributes = ["key1": "value1", "key2": "value2"]
// customData: Dictionary containing extra information that
application want to send to the server along with this event

hydra.reportUserSignOut(customData: customAttributes)
```

5. Current Screen

HydraGamification uses screen information to determine when and where campaigns should be displayed.

Swift (UIKit)

```
None
override func viewDidAppear(_ animated: Bool) {
    super.viewDidAppear(animated)

    hydra.gamificationScreenAppeared("SampleScreenName")
}

override func viewWillDisappear(_ animated: Bool) {
     super.viewWillDisappear(animated)
     hydra.gamificationScreenDisappeared("SampleScreenName")
}
```

SwiftUI

```
None
HydraGamificationUI().gamificationScreen("SampleScreenName")
Note:- The `gamificationScreen` modifier automatically reports
screen appearance and disappearance. No manual lifecycle callbacks
are required.
```

Objective-C

```
None
- (void)viewDidAppear:(BOOL)animated {
    [super viewDidAppear:animated];

    [HydraGamificationUI reportScreen:@"SampleScreenName"];
}
- (void)viewWillDisappear:(BOOL)animated {
     [super viewWillDisappear:animated];

    [HydraGamificationUI
reportScreenDisappeared:@"SampleScreenName"];

}
```

IMPORTANT

- Screen reporting is mandatory for screen-targeted campaigns.
- Campaign visibility and eligibility depend on the currently reported screen.

---

6. Custom Behavioral Events

Swift (UIKit)

```
None
let customAttributes
 = ["key1": "value1", "key2": "value2"]
hydra.reportBehavioralEvent(name:"custom_eventname",
customAttributes: customAttributes
)
```

7. Set Capillary Token (LoyaltyView)

A new public API has been introduced to support Loyalty View functionality.

Set the Capillary B2B short-lived token before displaying a Loyalty View.

Swift

```
None
hydra.setCapToken(token)
```

SwiftUI

```
None
hydra.setCapToken(token)
```

Objective-C

```
None
[HydraGamificationUI setCapToken:token];
```

IMPORTANT

- The Capillary token must be set before rendering the Loyalty View.
- Failure to provide a valid token may result in loyalty content not being displayed.

Automatic Experiences

Once configured, the following campaign types are automatically handled by Hydra.

- Floating Button Campaigns
- Popup Campaigns
- Bottom Sheet Campaigns

No additional implementation is required.

Hydra automatically evaluates campaign eligibility and displays content when applicable.

Custom UI Components

If you would like to display gamification content directly within your application screens, the following UI components are available.

For each view:

- Pass a unique view ID.
- Register a height callback.
- Update the view height dynamically.

Banner View

Swift (UIKit)

```
None
let bannerView = hydra.gamificationBannerView(
    bannerId: BANNER_ID
) { height in
    self.bannerHeightConstraint.constant = height
}
```

SwiftUI

```
None
GamificationBannerView(bannerId: BANNER_ID)
```

Objective-C

```
None
UIView *banner =
[HydraGamificationUI bannerViewWithBannerId:BANNER_ID
                                   onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Embedded View

Swift

```
None
let embedView = hydra.gamificationEmbedView(
    embedId: EMBED_ID
) { height in
    self.heightConstraint.constant = height
}
```

SwiftUI

```
None
GamificationEmbedView(
    embedId: EMBED_ID
)
```

Objective-C

```
None
UIView *embed =
[HydraGamificationUI embedViewWithEmbedId:EMBED_ID
                                 onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Loyalty View

Swift

```
None
let loyaltyView = hydra.gamificationLoyaltyView(
    embedId: LOYALTY_ID
) { height in
    self.heightConstraint.constant = height
```

```
}
```

SwiftUI

```
None
GamificationLoyaltyView(
    embedId: LOYALTY_ID
)
```

Objective-C

```
None
UIView *loyalty =
[HydraGamificationUI loyaltyViewWithEmbedId:LOYALTY_ID
                                   onHeight:^(CGFloat height) {
    self.heightConstraint.constant = height;
}];
```

Integration Checklist

Required

- Update to the latest Hydra iOS SDK version.
- Add `hydra_gamification_region`.
- Add `hydra_gamification_key`.
- Add `hydra_gamification_base_url` (Optional).
- Report screen names.
- Set the Capillary token using `setCapToken(...)` (Required for Loyalty View).

Optional

- Add Banner View.
- Add Embedded View.
- Add Loyalty View.

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

If you choose to integrate the optional UI components, Banner, Embedded, and Loyalty experiences can also be displayed directly within your application screens.

Android SDK Integration Guide (Without Fire…

Hydra Android SDK Integration Guide (Without Firebase)

Prerequisites

Before you get started, you need to register your app with the Capillary team. For registration, you need to provide the **application ID (package name)** for your Android app.

Our app team will register the app with our system and will provide you below details:

- Account Id
- Base URL
- SSL Public Key
- Org Id
- Gamification Key
- Gamification Region
- Gamification URL (Optional)

You will also need to get access for the below GitHub repository from our access team:

- https://github.com/Capillary/hydra-sdk-android-maven

Installation

Access to Hydra SDK Library

The Hydra SDK is hosted in a private GitHub Maven repository. You must have:

- A GitHub username and Personal Access Token (PAT) with `read:packages` scope.
- Access granted to the `Capillary/hydra-sdk-android-maven` repository.

Create a `github.properties` file at the **project root** (same level as `settings.gradle`):

```
None
username=YOUR_GITHUB_USERNAME
token=YOUR_GITHUB_PAT
```

Add it to `.gitignore`:

```
None
github.properties
```

Add GitHub Packages Maven Repository

Add the following to your `settings.gradle` inside `dependencyResolutionManagement.repositories`:

Groovy DSL (`settings.gradle`):

```
None
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven {
            def propsFile = file('github.properties')
            def props = new Properties()
            props.load(new FileInputStream(propsFile))

            name "GitHubPackages"
            url =
uri("https://maven.pkg.github.com/Capillary/hydra-sdk-android-maven")
            credentials {
                username = props['username']
                password = props['token']
            }
        }
    }
}
```

Kotlin DSL (`settings.gradle.kts`):

```
Kotlin
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven {
            val propsFile = file("github.properties")
```

```
            val props = java.util.Properties().apply {
load(propsFile.inputStream()) }

            name = "GitHubPackages"
            url =
uri("https://maven.pkg.github.com/Capillary/hydra-sdk-android-maven")
            credentials {
                username = props["username"] as String
                password = props["token"] as String
            }
        }
    }
}
```

Add Dependencies

Add Hydra SDK dependencies to `app/build.gradle`. Replace `<VERSION>` with the SDK version provided by the Capillary team.

> **Important:** Do NOT add the `com.google.gms.google-services` plugin. Do NOT add Firebase dependencies or `google-services.json`.

Groovy DSL:

```
None
dependencies {
    // Hydra Core (required)
    implementation "com.capillary:hydra-core:<VERSION>"

    // In-app messaging (optional)
    implementation "com.capillary:hydra-inapp:<VERSION>"

    // Inbox / Notification Center (optional)
    implementation "com.capillary:hydra-inbox-core:<VERSION>"
    implementation "com.capillary:hydra-inbox-ui:<VERSION>"

    // Gamification (optional)
    implementation "com.capillary:hydra-gamification:<VERSION>"
    implementation "com.capillary:hydra-gamification-ui:<VERSION>"
}
```

Kotlin DSL:

```kotlin
dependencies {
    // Hydra Core (required)
    implementation("com.capillary:hydra-core:<VERSION>")

    // In-app messaging (optional)
    implementation("com.capillary:hydra-inapp:<VERSION>")

    // Inbox / Notification Center (optional)
    implementation("com.capillary:hydra-inbox-core:<VERSION>")
    implementation("com.capillary:hydra-inbox-ui:<VERSION>")

    // Gamification (optional)
    implementation("com.capillary:hydra-gamification:<VERSION>")
    implementation("com.capillary:hydra-gamification-ui:<VERSION>")
}
```

1. Hydra SDK Version

Add the Hydra Android SDK version in your application.

Required Version: `1.9.22-beta02` (Beta for UAT/POC)

2. SDK Initialization

Initialize the Hydra SDK in your `Application.onCreate()` method.

Kotlin:

```kotlin
val hydraConfig = HydraConfig.Builder(
    applicationContext,
    accountId = accountId
)
    .orgId(orgId)
    .baseURL(baseUrl)
    .sslPublicKey(sslKey)
    .debugLevel(LogType.DEBUG)
    .gamificationWriteKey(gamificationWriteKey)
    .gamificationRegion(gamificationRegion)       // GamificationRegion.IN, .US,
or .ME
    .gamificationBaseUrl(gamificationBaseUrl)      // Optional
```

```
    .build()

HydraAPI.instanceWithConfig(applicationContext, hydraConfig)
```

Java:

```
Java
HydraConfig hydraConfig = new HydraConfig.Builder(
    getApplicationContext(), accountId
)
    .orgId(orgId)
    .baseURL(baseUrl)
    .sslPublicKey(sslKey)
    .debugLevel(LogType.DEBUG)
    .gamificationWriteKey(gamificationWriteKey)
    .gamificationRegion(gamificationRegion)
    .gamificationBaseUrl(gamificationBaseUrl)
    .build();

HydraAPI.instanceWithConfig(getApplicationContext(), hydraConfig);
```

Register your Application class in `AndroidManifest.xml`:

```
XML
<application
    android:name=".MyApplication"
    ... >
</application>
```

3. User Sign-In

Kotlin:

```
Kotlin
val hydraAPI = HydraAPI.getDefaultInstance(context)

val customAttributes = hashMapOf<String, Any>("key1" to "value1", "key2" to
"value2")
```

```
hydraAPI?.pushUserSignin(
    customerId,        // required – must match your Capillary CRM Account Linking
Identifier
    firstName,         // optional
    lastName,          // optional
    email,             // optional
    phone,             // optional
    customAttributes   // optional
)
```

Java:

```
Java
HydraAPI hydraAPI = HydraAPI.getDefaultInstance(context);

Map<String, Object> customAttributes = new HashMap<>();
customAttributes.put("key1", "value1");
customAttributes.put("key2", "value2");

if (hydraAPI != null) {
    hydraAPI.pushUserSignin(customerId, firstName, lastName, email, phone,
customAttributes);
}
```

4. User Sign-Out

Kotlin:

```
Kotlin
val hydraAPI = HydraAPI.getDefaultInstance(context)

hydraAPI?.pushUserSignout()
```

Java:

```
Java
HydraAPI hydraAPI = HydraAPI.getDefaultInstance(context);

if (hydraAPI != null) {
```

```
    hydraAPI.pushUserSignout();
}
```

5. Current Screen

HydraGamification uses screen information to determine when and where campaigns should be displayed.

Set up screen tracking in your `Application.onCreate()` after SDK initialization:

Kotlin:

```
Kotlin
var gamificationUIManager: GamificationUIManager? = null

HydraAPI.getDefaultInstance(applicationContext)?.getGamificationManager()?.let {
manager ->
    gamificationUIManager = GamificationUIManager(manager, applicationContext)
}

// Register activity lifecycle for screen tracking
registerActivityLifecycleCallbacks(object : ActivityLifecycleCallbacks {
    override fun onActivityResumed(activity: Activity) {
        gamificationUIManager?.onScreenChanged(activity,
activity::class.simpleName)
    }
    override fun onActivityDestroyed(activity: Activity) {
        gamificationUIManager?.onActivityDestroyed(activity)
    }
    override fun onActivityCreated(activity: Activity, savedInstanceState:
Bundle?) {}
    override fun onActivityStarted(activity: Activity) {}
    override fun onActivityPaused(activity: Activity) {}
    override fun onActivityStopped(activity: Activity) {}
    override fun onActivitySaveInstanceState(activity: Activity, outState: Bundle)
{}
})

// Register app foreground/background lifecycle
ProcessLifecycleOwner.get().lifecycle.addObserver(object :
DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) {
```

```
        gamificationUIManager?.onAppForegrounded()
    }
    override fun onStop(owner: LifecycleOwner) {
        gamificationUIManager?.onAppBackgrounded()
    }
})
```

IMPORTANT: Screen reporting is mandatory for screen-targeted campaigns. Campaign visibility and eligibility depend on the currently reported screen.

6. Custom Behavioral Events

Kotlin:

```kotlin
val hydraAPI = HydraAPI.getDefaultInstance(context)

val attributes = mapOf("key1" to "value1", "key2" to "value2")

hydraAPI?.pushEvent("custom_event_name", attributes)
```

Java:

```java
HydraAPI hydraAPI = HydraAPI.getDefaultInstance(context);

Map<String, Object> attributes = new HashMap<>();
attributes.put("key1", "value1");
attributes.put("key2", "value2");

if (hydraAPI != null) {
    hydraAPI.pushEvent("custom_event_name", attributes);
}
```

7. Set Capillary Token (Loyalty View)

A public API is available to support Loyalty View functionality. Set the Capillary B2B short-lived token before displaying a Loyalty View.

Kotlin:

```kotlin
val hydraAPI = HydraAPI.getDefaultInstance(context)

hydraAPI?.setCapToken(token)
```

Java:

```java
HydraAPI hydraAPI = HydraAPI.getDefaultInstance(context);

if (hydraAPI != null) {
    hydraAPI.setCapToken(token);
}
```

IMPORTANT: The Capillary token must be set before rendering the Loyalty View. Failure to provide a valid token may result in loyalty content not being displayed.

Automatic Experiences

Once configured, the following campaign types are automatically handled by Hydra:

- Floating Button Campaigns
- Popup Campaigns
- Bottom Sheet Campaigns

No additional implementation is required. Hydra automatically evaluates campaign eligibility and displays content when applicable.

Custom UI Components

If you would like to display gamification content directly within your application screens, the following UI components are available.

For each view:

- Add the view in your XML layout.
- Banner and Loyalty views are **automatically populated** by the SDK when `onScreenChanged()` is called.
- Embed views require explicit binding via `bindEmbedView()`.

Banner View

XML Layout:

```xml
<com.capillary.hydra.gamification.ui.entrypoints.BannerView
    android:id="@+id/bannerView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Banner views are automatically rendered by the SDK when `onScreenChanged()` reports the current screen. No manual binding is required.

Embed View

XML Layout:

```xml
<com.capillary.hydra.gamification.ui.entrypoints.EmbedWebView
    android:id="@+id/embedView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Kotlin:

```kotlin
val embedView = findViewById<EmbedWebView>(R.id.embedView)
gamificationUIManager?.bindEmbedView(embedView, EMBED_ENTRY_POINT_ID, this)
```

Loyalty View

XML Layout:

```xml
<com.capillary.hydra.gamification.ui.entrypoints.LoyaltyEmbedView
    android:id="@+id/loyaltyView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Loyalty views are automatically rendered by the SDK when `onScreenChanged()` reports the current screen. Ensure `setCapToken(...)` is called before the screen is reported.

Integration Checklist

Required

- ☐ Add `github.properties` with credentials and add to `.gitignore`
- ☐ Add GitHub Packages Maven repo to `settings.gradle`
- ☐ Add `hydra-core` dependency to `app/build.gradle`
- ☐ Update to the latest Hydra Android SDK version
- ☐ Initialize SDK in `Application.onCreate()` via `HydraConfig.Builder`
- ☐ Register Application class in `AndroidManifest.xml`
- ☐ Add `hydra_gamification_region`
- ☐ Add `hydra_gamification_key`
- ☐ Add `hydra_gamification_base_url` (Optional)
- ☐ Report screen names via `GamificationUIManager`
- ☐ Set the Capillary token using `setCapToken(...)` (Required for Loyalty View)
- ☐ Ensure **no** `google-services` plugin, Firebase dependencies, or `google-services.json`

Optional

- ☐ Add In-App Messaging (`hydra-inapp`)
- ☐ Add Inbox / Notification Center (`hydra-inbox-core`, `hydra-inbox-ui`)
- ☐ Add Banner View
- ☐ Add Embed View
- ☐ Add Loyalty View
- ☐ CI/CD: Generate `github.properties` from secrets before Gradle build

Result

After completing the required steps, HydraGamification campaigns such as:

- Floating Buttons
- Bottom Sheets
- Popups

will automatically become available within your application.

If you choose to integrate the optional UI components, Banner, Embed, and Loyalty experiences can also be displayed directly within your application screens.

Push notifications are not available in this integration mode. To add push notification support later, include the `hydra-pushbase`, `hydra-firebase` modules along with Firebase dependencies and `google-services.json`, and follow the standard integration guide.